



AGENTIC SDLC: THE SOFTWARE FACTORY FRAMEWORK

DESCRIPTION	Complete framework for industrializing software development with AI agents. Defines how agents serve as the logistic layer while humans design, govern, and optimize the production flow.
STATUS	LIVE
LAST UPDATED	May 20, 2026
TAGS	Core, SDLC, Methodology, Industrialization
RELATED	context-engineering , model-context-protocol , context-gates , levels-of-autonomy , model-routing , the-spec , ai-amplification , agent-optimization-loop , provenance , extreme-programming , theory-of-llm-constraints

1. DEFINITION

The Agentic Software Development Life Cycle (ASDLC) is a framework for industrializing software engineering. It represents the shift from craft-based development (individual artisans, manual tooling, implicit knowledge) to industrial-

scale production (standardized processes, agent orchestration, deterministic protocols).

“Agentic architecture is the conveyor belt for knowledge work.” – Ville Takanen

ASDLC is not about “AI coding assistants” that make developers 10% faster. It’s about building the **software factory**—systems where agents serve as the architecture of labor while humans design, govern, and optimize the flow.

2. THE INDUSTRIAL THESIS

Agents do not replace humans; they industrialize execution.

Just as robotic arms automate welding without replacing manufacturing expertise, agents automate high-friction parts of knowledge work (logistics, syntax, verification) while humans focus on intent, architecture, and governance.

In this model:

- **Agents are the logistic layer** — Moving information, verifying specs, executing tests
- **Context is the supply chain** — Just-in-Time delivery of requirements, schemas, and code
- **Standardization is mandatory** — Schemas, typed interfaces, deterministic protocols replace “vibes”
- **AI Amplification** — Agents act as a “High-Pass Filter” for process maturity: they accelerate good practices but amplify the chaos of bad ones.
- **Provenance is Value** — In a world of infinite code, value shifts to Provenance: the audit trail of who steered the agent and how it was verified.

3. THE CYBERNETIC MODEL

ASDLC operates at L3 Conditional Autonomy—a “Fighter Jet” model where the Agent acts as the Pilot executing maneuvers, and the Human acts as the Instructor-in-the-Cockpit.

Key Insight: Compute is cheap, but novelty and correctness are expensive. Agents naturally drift toward the “average” solution (Regression to the Mean). The Instructor’s role is not to write code, but to define failure boundaries (Determinism) and inject strategic intent (Steering) that guides agents out of mediocrity.

4. THE CYBERNETIC LOOP

The lifecycle replaces the linear CI/CD pipeline with a high-frequency feedback loop:

Mission Definition: The Instructor defines the “Objective Packet” (Intent + Constraints). This is the core of Context Engineering.

Generation (The Maneuver): The Agent autonomously maps context—often using the Model Context Protocol (MCP) to fetch live data—and executes the task.

Verification (The Sim): Automated Gates check for technical correctness (deterministic), while the Agent’s Constitution steers semantic intent (probabilistic).

Course Correction (HITL): The Instructor intervenes on technically correct but “generic” solutions to enforce architectural novelty.

5. STRATEGIC PILLARS

FACTORY ARCHITECTURE (ORCHESTRATION)

Projects structured with agents as connective tissue, moving from monolithic context windows to discrete, specialized stations (Planning, Spec-Definition, Implementation, Review).

STANDARDIZED PARTS (DETERMINISM)

Schema-First Development where agents fulfill contracts, not guesses. `AGENTS.md` , `specs/` , and strict linting serve as the “jigs” and “molds” that constrain agent output.

QUALITY CONTROL (GOVERNANCE)

Automated, rigorous inspection through Probabilistic Unit Tests and Human-in-the-Loop (HITL) gates. Trust the *process*, not just the output.

THE AGENT FACTORY (META-OPTIMIZATION)

The underlying machinery that builds the agents themselves. While the Ralph Loop produces code, the Agent Optimization Loop produces better agents by testing them against **Scenarios** rather than static benchmarks.

6. ASDLC USAGE

Full project vision: </docs/vision.md>

Applied in: [Specs](#), [AGENTS.md Specification](#), [Context Gates](#), [Model Routing](#), [Agent Optimization Loop](#)

// ASDLC.IO

Definitive knowledge base
for Agentic Software
Development Life Cycle

COMMUNITY

GITHUB

DISCORD

EXPLORE

RECIPES

MIT LICENSE

patterns, practices, and
recipes.

© 2026 ASDLC.IO. ALL RIGHTS RESERVED.

RELEASE V0.27.1